

DESIGN PROPOSAL · BUILD TRACK

Open Clearinghouse Blueprint

A small, vendor-neutral engine for walletless network spend. One process, one database, seven ports. Pymthouse becomes one set of adapters plus a product on top, not the reference.

draft v1 · 2026-09-09

grounded in the Pymthouse main review

contract-first · hexagonal

1 Thesis 2 Core and ports 3 Authorize, then settle 4 The three contracts
5 Where Pymthouse fits 6 Operator modes and liability 7 Migration path
8 Decisions and non-goals

1 · THESIS

Standardize the boundary Pymthouse already has, then shrink everything else

The Pymthouse review showed a clean shape hiding under a large product: a policy and identity service that decides who may spend, a signer that asks it before every ticket, and an asynchronous usage stream that becomes charges. That shape is the clearinghouse. Everything that makes Pymthouse hard to run elsewhere is vendor plumbing around it: a full OpenID Connect issuer, hosted OpenMeter, two Stripe webhook planes, Turnkey custody, a settlement service and a support console.

The advice is therefore not to design a new engine from a blank page. It is to lift three contracts out of Pymthouse, put a small core behind them, ship minimal adapters that need nothing but Postgres, and let Pymthouse re-implement the adapters with its commercial integrations. The open reference path and the commercial path then share one core and one conformance suite, which is what makes the result vendor-agnostic in practice rather than on paper.

Contract 1 · Authorize

The signer-to-clearinghouse protocol. Already exists as the identity webhook package. Becomes the core's primary inbound API.

Contract 2 • Usage event

One idempotent event per signed ticket. Already exists implicitly in the collector mapping. Becomes the schema every adapter speaks.

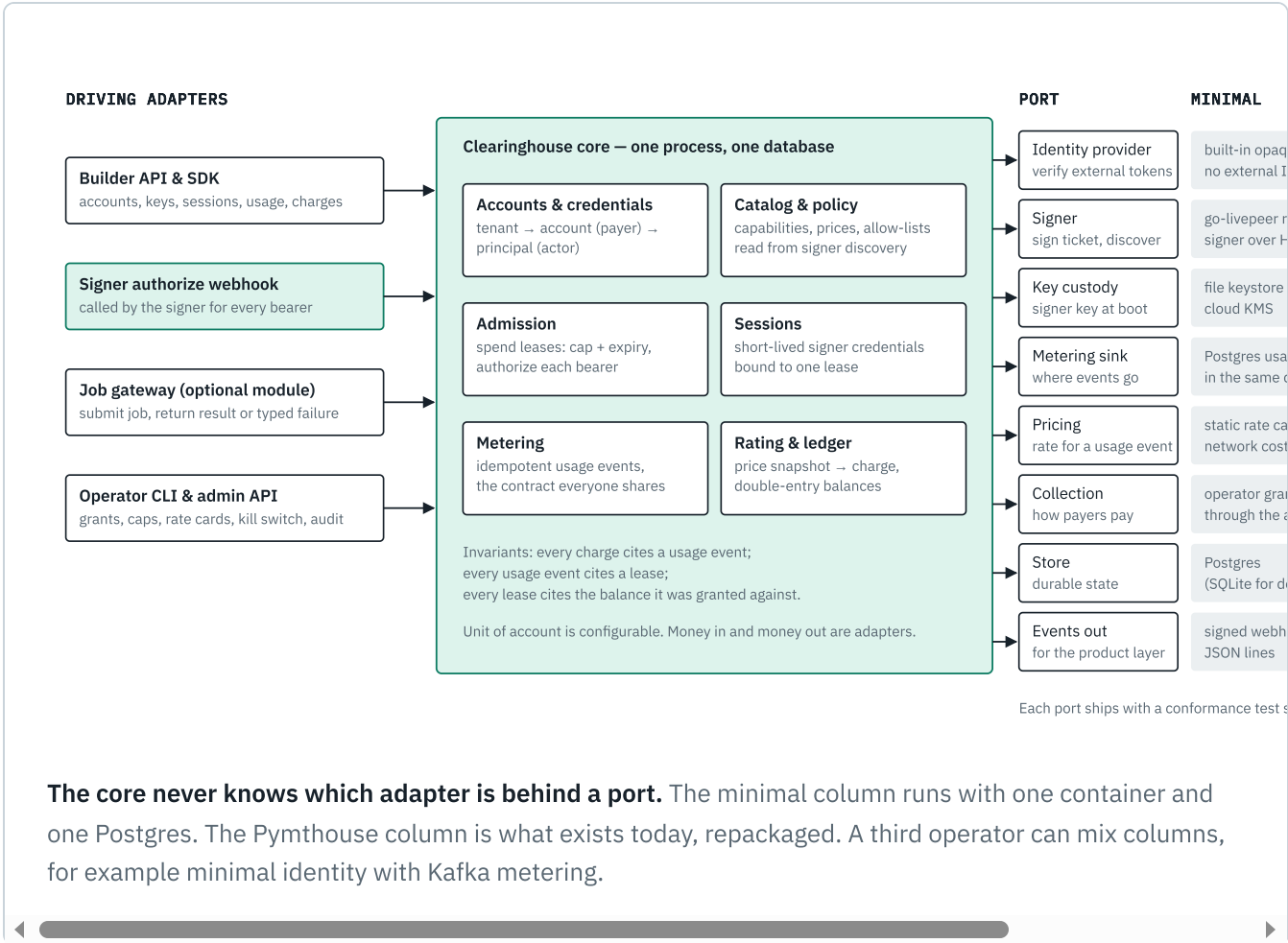
Contract 3 • Port interfaces

Seven small interfaces with a conformance test kit. New. This is what lets Livepeer Inc, BlueClaw and Pymthouse run interchangeable instances.

2 • COMPONENT DIAGRAM

The core and its ports

Hexagonal layout. Callers on the left drive the core through a versioned HTTP API. The core owns six responsibilities and talks to the outside world only through ports on the right. Each port has a minimal adapter that needs nothing but the database, and a Pymthouse adapter that plugs in what runs today.



core or primary inbound contract component

API surface, versioned and small

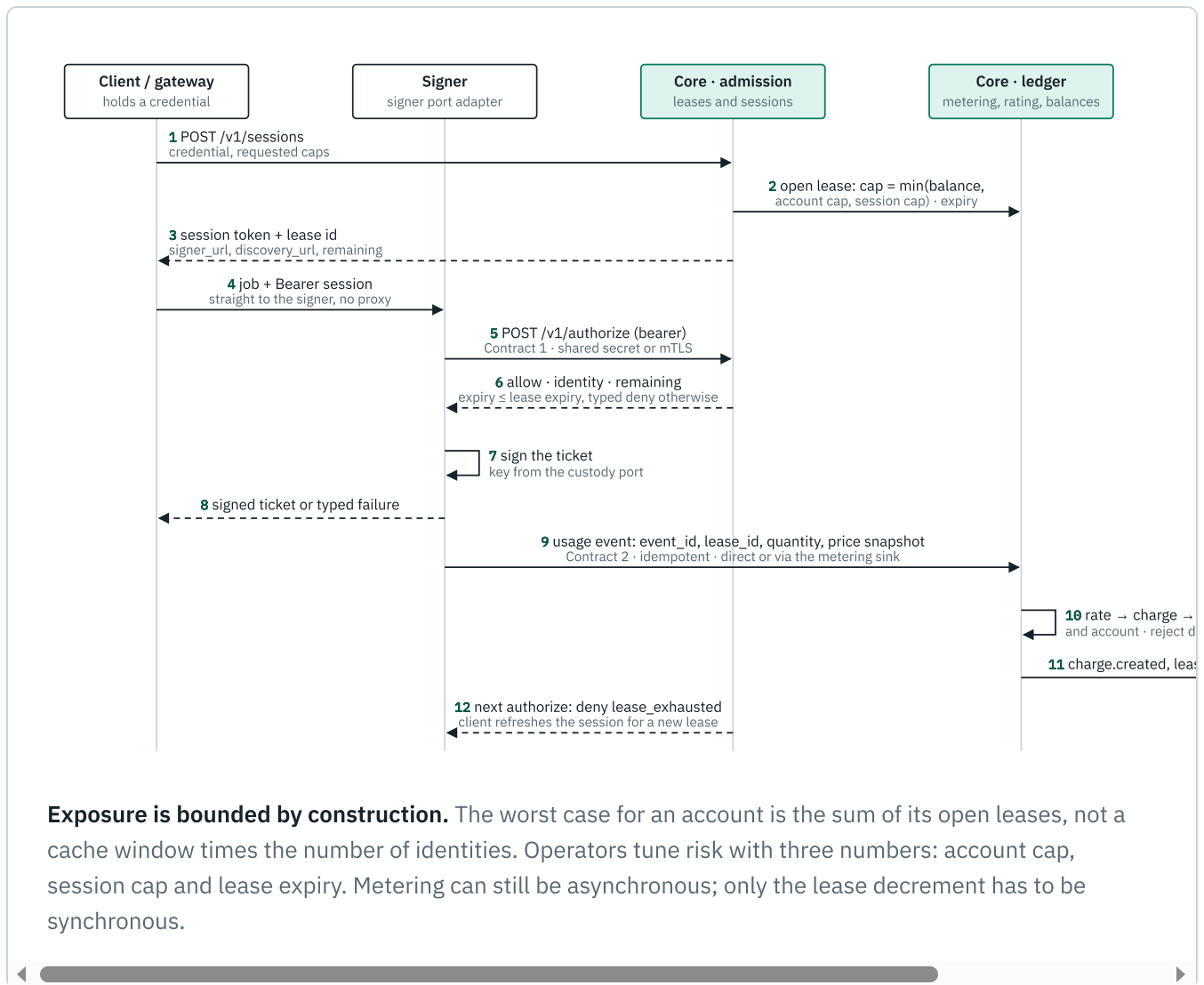
RESOURCE	VERBS	WHO	NOTES
/v1/tenants , /v1/accounts	create, read, cap, suspend	operator, tenant admin	Account is the payer. Principals hang off accounts.
/v1/credentials	issue, rotate, revoke	tenant admin, self-serve	Opaque prefixed keys, hashed at rest. External JWTs accepted through the identity port.
/v1/catalog	list capabilities and prices	anyone with a credential	Shaped by the account's policy. Cached from signer discovery.
/v1/sessions	mint, refresh	credential holder	Returns signer URL, discovery URL, short-lived session and the lease it is bound to.
/v1/authorize	post	signer only, shared secret or mTLS	Contract 1. Returns allow or a typed deny, the usage identity and an expiry.
/v1/usage	ingest, query	signer or collector; tenants read	Contract 2. Idempotent on event id. Query by account, principal, job, time window.
/v1/charges , /v1/balances	read	tenants, operator	Every charge links to its usage event and price snapshot.
/v1/grants	credit, debit, adjust	operator, collection adapter	The only way money enters the ledger. Idempotent, audited.
/v1/jobs	submit, status, result	credential holder	Optional module. Wraps a bundled gateway for builders who run nothing else.

3 · DATA FLOW DIAGRAM

Authorize, then settle

Pymthouse checks a cached balance and lets cost arrive later, so an account can drift negative by one cache window per identity. The engine should make that bound explicit with a lease. A session is granted a spend lease with a cap and an expiry. The signer's authorize calls are answered against the lease, usage events settle against it, and the

ledger's exposure is never more than the sum of open leases.



4 · CONTRACTS

The three contracts that make it vendor-agnostic

Contract 1 · Authorize

Lift the existing identity webhook protocol into the engine, drop Pymthouse-specific claims from the wire, and add the lease. Any signer that can make one HTTP call per bearer can participate. Any clearinghouse that implements it can sit behind go-livepeer.

```

POST /v1/authorize
{ "bearer": "...", "signer_id": "sig_...", "context": { "capability": "...", "estimate": "..."}

200 { "decision": "allow",
      "identity": { "tenant": "t_...", "account": "acct_...", "principal": "p_..." },
      "lease": { "id": "lease_...", "remaining": "...", "unit": "usd_micros" },
      "expiry": 60
    }
  
```

9/9/26, 3:31 PMOpen Clearinghouse Blueprint

expires_in : 00 s

402 { "decision": "deny", "reason": "lease_exhausted" | "account_suspended" | "unknown_"

503 { "decision": "deny", "reason": "ledger_unavailable" } # fail closed

Contract 2 • Usage event

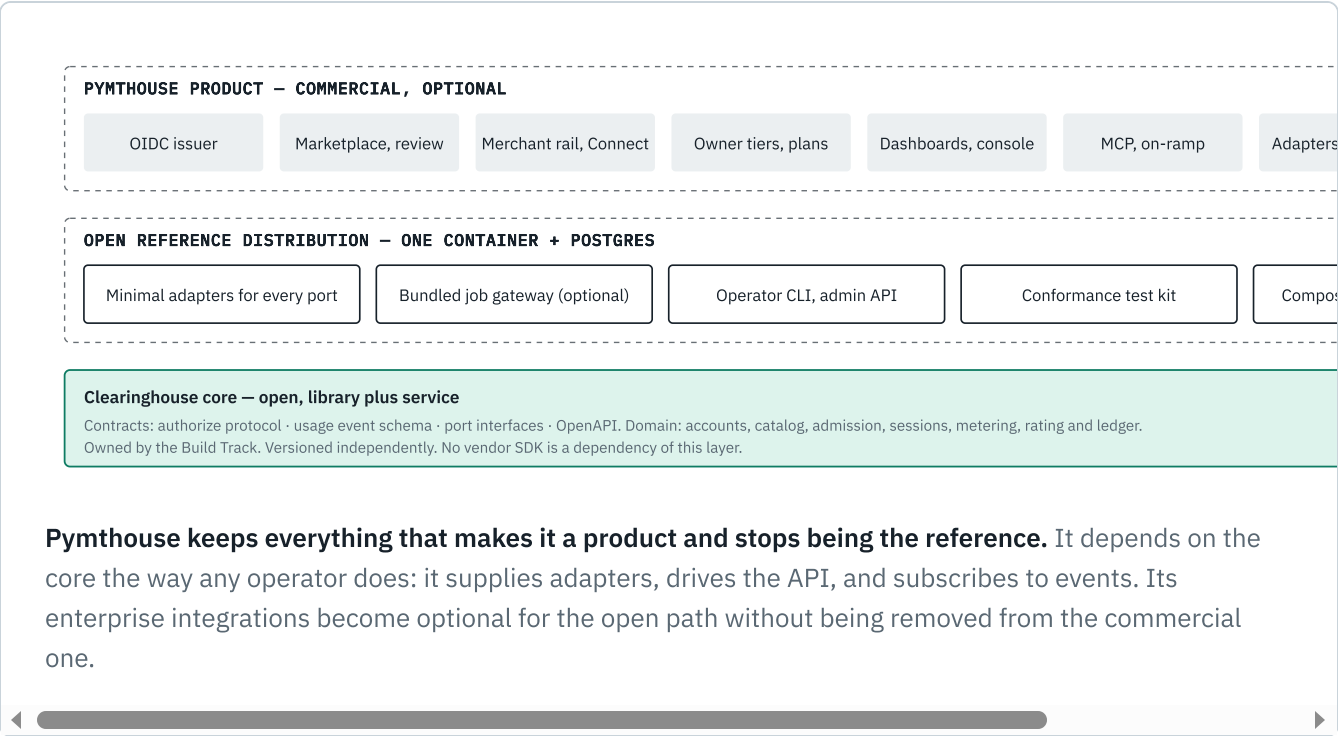
One event per signed ticket or per job step. The collector mapping in Pymthouse already contains every field; write it down as a schema, version it, and require idempotency on the event id.

FIELD	MEANING	PYMTHOUSE TODAY
event_id	Idempotency key from the producer	gateway request id or ticket id
lease_id	The lease this settles against	none; new
tenant , account , principal	Who owns it, who pays, who acted	client_id , payer, actor split on #
job_id , manifest_id	Per-job attribution	manifest_id , session_id
capability , model	What was bought	pipeline , app
quantity + unit	Billable amount in the network's unit	fee_wei , billable_secs
price_snapshot	Rate used and its source, so charges are reproducible	eth_usd_price
producer , schema_version	Which signer, which version	implicit

Contract 3 • Port interfaces and conformance

Keep each port to a handful of methods. Publish a test kit that spins the core against an adapter and asserts behavior: idempotent ingest, deny on empty balance, key never written to disk unencrypted, event ordering. Adapters live in their owners' repositories and claim conformance by running the kit in CI. This is the mechanism that keeps a Livepeer Inc instance, a BlueClaw instance and a Pymthouse instance interchangeable.

Three layers, two owners



Mapping today's Pymthouse onto the ports

PYMTHOUSE PIECE	BECOMES	EFFORT SIGNAL
Identity webhook package	Contract 1 reference implementation inside the core	Low. The package already exists and is versioned.
Collector mapping	Contract 2 schema plus the Kafka metering-sink adapter	Low. Fields already exist; add lease id and schema version.
Mint and balance gate	Sessions and admission in the core, with leases	Medium. Replaces cache TTL logic with lease arithmetic.
OIDC issuer, three sibling clients	Identity-provider adapter that verifies its own JWTs	Medium. Issuer stays in Pymthouse; the core only verifies.
OpenMeter customers, plans, entitlements	Pricing adapter and an optional ledger mirror	High. Decide whether OpenMeter or the core ledger is source of truth per deployment.
Stripe planes A and B, settlement service	Collection adapter that posts grants and reads charges	Medium. Grants become the only write into the ledger.

Turnkey keystore and funding webhook	Custody adapter plus a treasury extension	Low. Already isolated in the container endpoint.
Dashboards, marketplace, MCP, console	Product layer over the API and events out	Medium. Read models move from Postgres and OpenMeter to the API.

6 · SUSTAINABILITY AND LIABILITY

Make custody a choice the operator makes on purpose

The concern raised in the meeting is real: a walletless clearinghouse in production holds value and personal data, so it needs an operator with a revenue model, a security posture and a liability boundary. The engine cannot supply the operator, but it can make the boundary explicit and let a deployment choose how much it holds.

Mode 1 · Identity and metering only

The builder brings their own signer and wallet. The core issues credentials, authorizes, meters and reports. It holds no funds. Liability is limited to correctness of records. Good fit for Livepeer Inc behind an internal product.

Mode 2 · Operator-funded prepaid

The operator's signer pays orchestrators. Builders prepay through grants. Exposure equals the ticket-broker deposit and reserve plus the sum of open leases, all visible on one admin page. This is the evaluation deployment a Cloud SPE instance could run with a stated cap.

Mode 3 · Full product

Mode 2 plus card collection, merchant resale, tiers and support. This is Pymthouse. The revenue model and the liability sit with its operator, and the open reference path never depends on it.

Controls the core should ship with, not leave to adapters

- **Global exposure cap.** The core refuses to open a lease when the sum of open leases would exceed a configured limit. The operator can state their maximum loss in one number.
- **Kill switch.** One admin call sets every authorize answer to deny with a stated reason. Existing

leases expire naturally.

- **Fail closed.** If the ledger or identity port is unavailable, authorize denies with a distinct reason so signers and clients can tell an outage from a refusal.
- **Pseudonymous by default.** The core stores opaque ids and hashed credentials. Names, emails and payment details live in the product layer behind the events-out port, which keeps the reference deployment out of most personal-data obligations.
- **Append-only audit.** Grants, caps, kill-switch changes and adapter configuration changes are recorded with an actor and reason and exported through events out.

7 • MIGRATION

Strangler path from Pymthouse main

1. **Write the contracts down.** Publish Contract 1 and Contract 2 as versioned schemas extracted from the webhook package and the collector mapping. Pymthouse adopts them with no behavior change. This is a documentation and packaging release.
2. **Build the core against the minimal adapters.** Accounts, credentials, sessions, admission with leases, metering, rating, ledger, admin API. Ship the compose file and the conformance kit. Run it standalone against a signer in Mode 1.
3. **Shadow it inside Pymthouse.** Point a second consumer of the Kafka topic at the core's usage ingest and compare charges to OpenMeter for a few weeks. Differences are either bugs or pricing-adapter gaps, and both are useful to find before cut-over.
4. **Move authorize and mint.** Pymthouse's webhook route delegates to the core with the OIDC identity adapter. Balance gate TTLs are replaced by leases. This removes the eventual-consistency finding from the review.
5. **Decide the ledger's source of truth per deployment.** Pymthouse may keep OpenMeter authoritative and mirror into the core, or flip. The reference distribution never needs OpenMeter either way.
6. **Retire duplicated read models.** Dashboards and console read from the API and events out. Legacy ledger tables in Pymthouse can be dropped once nothing reads them.